Sistemas Digitales — Semana 1, Parte 1

Sistema embebido y microcontrolador

Un sistema embebido realiza una función específica. Generalmente está dentro de otro equipo, como electrodomésticos, sensores, autos, etc.

Microcontrolador: integra CPU, memoria y periféricos de entrada/salida en un solo chip. Se usa para controlar tareas concretas.

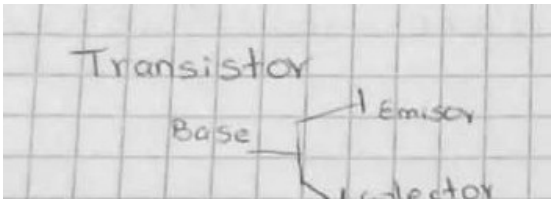
Microprocesador: tiene bastante capacidad de cálculo, se usa en computadoras, laptops y servidores. Normalmente necesita memoria y otros componentes externos. Consume más recursos según el sistema.

En cambio el microcontrolador suele gastar menos energía y tiene recursos más limitados. Puede trabajar ejecutando un programa de control de forma continua.

Transistor

Es uno de los componentes básicos de la electrónica digital. Con transistores se forman compuertas lógicas, y estas se utilizan dentro de circuitos como la ALU.

En clase anotamos sus terminales: base, emisor y colector (para un transistor BJT).



Partes principales de un computador

- CPU: ejecuta las instrucciones y realiza operaciones.
- Memoria: guarda el programa y los datos.
- Entrada/salida: permite comunicarse con botones, sensores, pantallas y otros dispositivos.

Dentro de la CPU

- ALU: unidad aritmético-lógica, operaciones como sumas, AND, OR, etc.
- Registros: pequeñas posiciones rápidas dentro de la CPU.
- Unidad de control: coordina las instrucciones; también está el contador de programa.

La CPU sigue el ciclo: buscar → decodificar → ejecutar y vuelve a repetirlo.

Sistemas Digitales — Semana 1, Parte 1

Buses

Los buses son líneas que permiten comunicar CPU, memoria y periféricos. Se utilizan para transportar información y controlar las operaciones.

- Bus de datos: lleva los bits que se leen o escriben. Puede ser bidireccional.
- Bus de direcciones: indica la posición de memoria con la que se va a trabajar.
- Bus de control: señales para coordinar lectura, escritura y otras operaciones.

Por ejemplo, para leer un dato la CPU señala una dirección, activa la lectura y recibe el contenido por el bus de datos.

¿Qué es la memoria?

Es donde se almacenan bits, representados con 0 y 1. Cada posición tiene una dirección para poder acceder a lo que está guardado.

RAM: volátil; pierde la información cuando se corta la energía. Se usa mientras se ejecuta el programa.

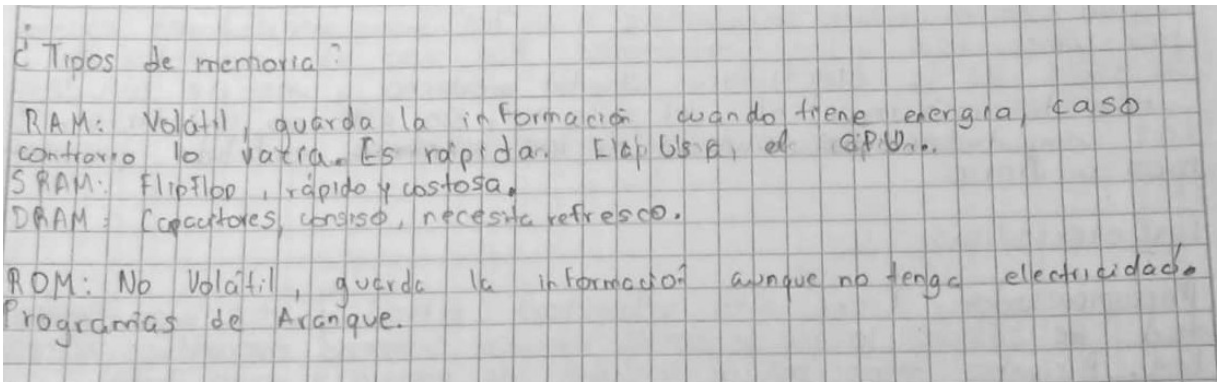
- SRAM: rápida, utiliza circuitos tipo flip-flop. Más costosa.
- DRAM: guarda bits mediante capacitores y necesita refrescar su contenido.

ROM: no volátil, conserva la información sin electricidad. Se relaciona con programas de arranque y datos fijos.

EPROM: memoria no volátil que puede borrarse con luz ultravioleta y volver a programarse.

Flash: memoria no volátil. Guarda el programa del microcontrolador; al apagarlo no se pierde.

EEPROM: también conserva datos sin energía, útil para configuraciones pequeñas que deban mantenerse.



Unidades y una diferencia importante

1 byte = 8 bits. 1 KB = 1024 bytes. La Flash guarda el código; la SRAM guarda variables y datos de trabajo mientras el programa está funcionando.

También vimos Harvard (separa memoria/caminos de programa y datos) y von Neumann (comparten el camino o memoria, según la arquitectura).

Sistemas Digitales — Semana 1, Parte 2

Bits, bytes y sistemas numéricos

Un bit puede representar 0 o 1, que en un pin digital corresponden a niveles de voltaje bajo o alto. En una placa AVR de 5 V, el alto se aproxima a 5 V; en el ESP32 es de 3,3 V.

Un byte son 8 bits. Los bits se cuentan desde la derecha: bit 0 hasta bit 7. En hexadecimal, cada dígito representa 4 bits, se usan 0–9 y A–F.

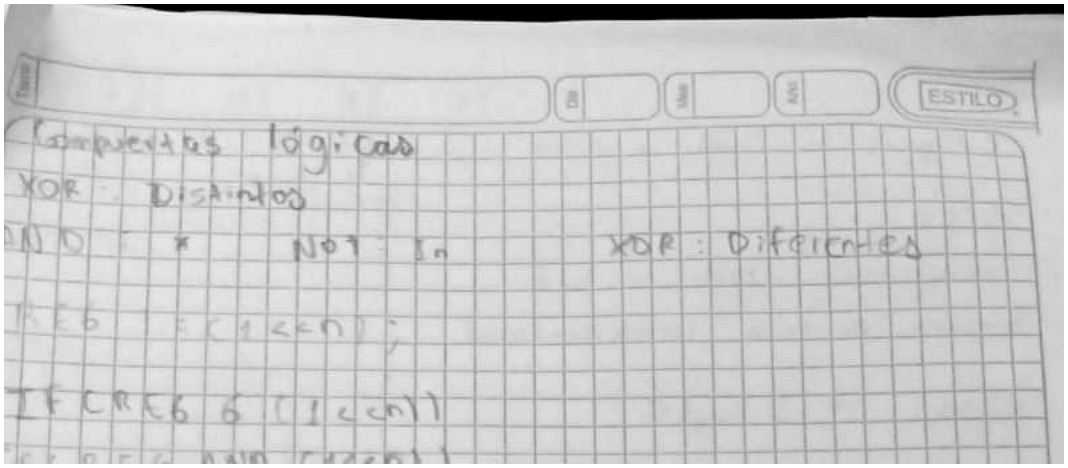
- Ejemplo: 0x81 = 10000001 en binario = 129 en decimal.
- 0xFF = 255 y 0x08 = 00001000, aquí está encendido el bit 3.
- Para escribir hexadecimal se utiliza el prefijo 0x. En C, para binario se puede utilizar 0b.

Operaciones bit a bit

Se aplican sobre cada posición del número. Las principales son:

- AND (&): el resultado es 1 si los dos bits son 1.
- OR (|): si alguno tiene 1, el resultado es 1.
- XOR (^): sale 1 cuando los bits son diferentes.
- NOT (~): invierte los bits.

También vimos los desplazamientos. 1 << 4 coloca un 1 en la posición 4 (00010000). Sirve para crear una máscara y modificar solo un bit.



Cambiar un bit sin tocar los demás

REG |= (1 << n); pone el bit n en 1. REG &= ~(1 << n); lo pone en 0. REG ^= (1 << n); invierte ese bit.

Si queremos saber si un bit está activo, usamos REG & (1 << n). No se cambia el valor de los demás bits.

Sistemas Digitales — Semana 1, Parte 2

Registros y pines de entrada/salida

En el microcontrolador los registros sirven para configurar y controlar los pines. Anotamos principalmente tres registros del AVR: DDRB, PORTB y PINB.

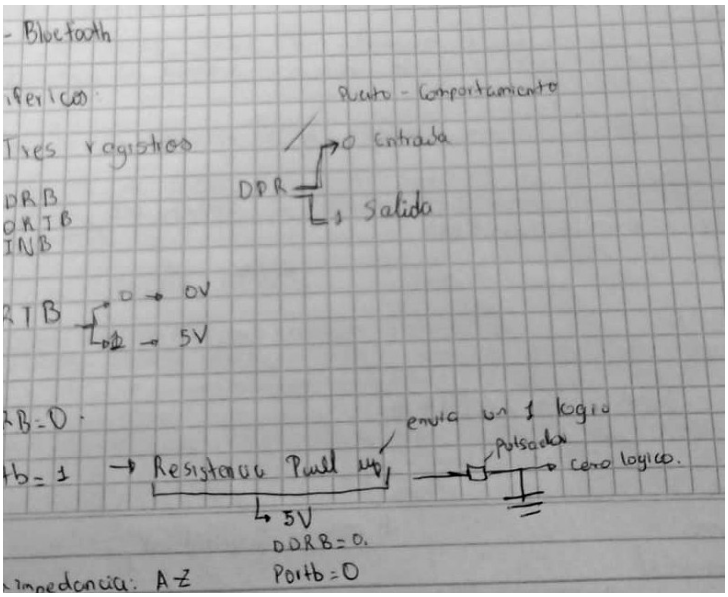
- DDRB: define si el pin es entrada (0) o salida (1).
- PORTB: cuando es salida, escribe 0 o 1; si es entrada, puede activar la resistencia pull-up interna.
- PINB: permite leer el estado lógico que hay en el pin.

Ejemplo: si DDRB5 = 1 y PORTB5 = 1, PB5 es salida en alto. Si PORTB5 = 0, se pone en bajo.

Entrada flotante y pull-up

Si DDRB = 0 en un bit y PORTB = 0, la entrada queda sin pull-up interna y puede estar flotante si no hay un circuito externo que fije el nivel.

Con DDR = 0 y PORT = 1, se activa el pull-up. Si conectamos un pulsador a GND, normalmente se lee 1 cuando está suelto y 0 cuando está presionado.



Ejercicios y código

Para apagar el bit 4 de PORTB sin afectar los demás:

```
PORTB &= ~(1 << 4);
```

Si DDRD2 = 0 y PORTD2 = 1, el pin D2 queda como entrada con pull-up. Al pulsar un botón conectado a GND, PIND2 lee 0.

La relación es sencilla: DDR configura, PORT escribe o activa pull-up, y PIN lee. Es lo que ocurre internamente cuando en Arduino usamos pinMode(), digitalWrite() y digitalRead().